

Fixture Layout Optimization in Wood Industry: from Constraint Solving and Reinforcement Learning to Particle Swarm Optimization

Anna Vitali^{a,*}, Roberto Amadini^a, Vittorio Maniezzo^a, Maurizio Gabbriellini^a,

^a*Department of Computer Science and Engineering, University of Bologna, Bologna, Italy*

Abstract

In the wood industry, the layout of fixtures supporting the workpiece has a critical impact on the quality of the final product. Vacuum suction cup fixtures, which can freely rotate, are commonly used to ensure workpiece stability during machining. However, determining an optimal fixture layout that satisfies the problem constraints while minimizing vibrations and oscillations is a challenging and time-consuming task when performed manually. This paper addresses the problem of optimally positioning rotatable fixtures by combining and comparing different optimization techniques. First, we solve a restricted version of the problem that neglects fixture rotations, comparing solutions obtained through Constraint Programming and Reinforcement Learning to generate initial feasible layouts. These layouts are then refined using a Particle Swarm Optimization algorithm that explicitly accounts for fixture rotation. Experimental results show that the proposed approach effectively supports expert operators in identifying high-quality fixture layouts, and that the quality of the initial solution significantly influences the final result, with Constraint Programming consistently providing better starting solutions than Reinforcement Learning.

© 2011 Published by Elsevier Ltd.

Keywords: Fixture layout optimization, constraint programming, reinforcement learning, meta-heuristics, wood industry

1. Introduction

In wood industry, finished products are typically obtained through machining processes in which a semi-finished component, such as a wood panel, is processed using a machine known as a *work center*, shown in Fig. 1a.

A typical work center is equipped with a tool storage system from which the tools required to perform operations such as cutting, milling, and contouring on the panel are automatically selected. However, the use of these tools can induce vibrations and oscillations in the workpiece, thereby reducing the quality of the final product [1, 2]. For this reason, keeping the workpiece as stable and firm as possible during machining is crucial.

The work center is equipped with a worktable featuring movable bars that translate along the machine's horizontal axis, above which vacuum suction cup fixtures can be positioned.

The fixtures can be square or rectangular in shape (Fig. 1b) and are composed of a square base, which allows them to be mounted on the bars and to slide along their length. To improve product quality through an appropriate fixture layout, the fixtures can also be freely rotated to achieve better alignment with the shape of the workpiece.

*Corresponding author

Email addresses: anna.vitali7@unibo.it (Anna Vitali), roberto.amadini@unibo.it (Roberto Amadini), vittorio.maniezzo@unibo.it (Vittorio Maniezzo), maurizio.gabbriellini@unibo.it (Maurizio Gabbriellini)

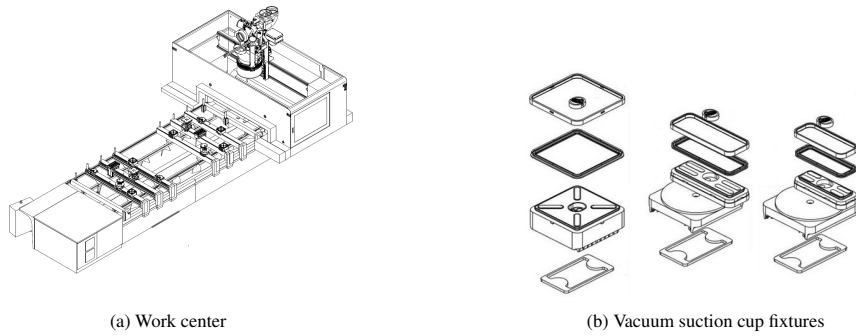


Figure 1. On the left, a work center featuring a worktable with movable bars. On the right, fixtures used to secure the workpiece during machining operations.

Determining an appropriate fixture layout for a given workpiece is not always straightforward for a human operator, especially when the workpiece has contoured shapes or includes areas to be extruded. Layouts are often based on the operator's experience; however, such manually designed solutions are not necessarily optimal and can often be improved by considering the structural properties of the fixtures system, such as its resistance to bending and torsional forces.

In this paper, we aim to determine a sound and possibly optimal placement of fixtures beneath the workpiece by identifying, the type, position, and rotation angle of the fixtures that can be used to sustain it. Our approach consists of two stages. First, we compute a feasible, but generally sub-optimal, placement by solving a simplified version of the original problem in which fixture rotations are neglected. To this end, we compare the quality of the solutions obtained using a Constraint Programming (CP) solver and a Reinforcement Learning (RL) algorithm. Second, the resulting layouts are refined through a local search procedure based on Particle Swarm Optimization (PSO), which explicitly accounts for fixture rotations.

This work extends [3], which proposed a hybrid approach combining Constraint Programming (CP), Mixed Integer Programming (MIP), and Particle Swarm Optimization (PSO) to solve the fixture layout optimization problem. We reformulated several constraints of the CP model, improving its performance on the most challenging instances for the considered solvers, and extend the experimental evaluation by introducing additional real-world case studies. Additionally, we propose a Q-learning-based algorithm for solving the simplified version of the problem and compare its performance with those of the CP and MIP models.

Experimental results show that the solutions produced by the CP and MIP models are of higher quality than those obtained using Q-learning and consistently outperform those designed by an expert operator. Starting from any of these initial layouts, PSO further improves the solution by increasing the moments of inertia of the fixture configuration, thereby enhancing workpiece stability.¹

Paper structure. The paper is organized as follows. Section 2 reviews the related work. Section 3 presents the background context. Section 4 describes the problem parameters and constraints. Section 5 introduces the CP model, while Section 6 presents the MIP model. Section 7 introduces the Q-learning based algorithm. Section 8 describes the PSO-based algorithm. Finally, Section 9 discusses the experimental results, and Section 10 concludes the paper.

2. Related Work

The problem of determining a fixture layout that maximizes workpiece stability has been widely studied in the literature. One class of approaches formulates fixture layout as a mathematical optimization problem, often coupled with Finite Element Analysis (FEA) [4] to evaluate workpiece deformation. Menassa and DeVries [5] proposed the use of the Broyden–Fletcher–Goldfarb–Shanno (BFGS) optimization algorithm to determine fixture support locations that

¹This work was carried out in collaboration with a leading company in the woodworking machinery sector. The name of the company is omitted for privacy reasons.

minimize deformation. This direction was further explored by Cai et al. [6], who combined nonlinear programming with FEA to optimize fixture layouts under a given force, while Camelio et al. [7] later extended the same approach to minimize shape variations in assembled products. Li and Melkote [8] applied Sequential Quadratic Programming (SQP) to jointly optimize fixture layout and clamping forces. Subsequently, Li et al. [9] proposed a flexible fixture system for sheet metal assembly based on assembly variation analysis using the Lagrangian conditional extremum method, while Zhang et al. [10] developed a nonlinear optimization model solved using the Augmented Lagrangian method.

Evolutionary and meta-heuristic optimization techniques have also been widely investigated. Kulankara et al. [11] proposed a Genetic Algorithm (GA) for the simultaneous optimization of fixture positions and clamping forces. Shortly afterwards, Y. Gene Liao [12] introduced a two-phase GA to identify and refine locating and clamping zones in sheet-metal assembly. Padmanaban et al. [13] combined FEA with discrete and continuous Ant Colony Algorithms (ACA) to reduce sheet metal deformation. Sundararaman et al. [14] proposed a fixture layout optimization approach combining Response Surface Methodology (RSM), Genetic Algorithms (GA), and Particle Swarm Optimization (PSO), reporting superior performance with PSO. More recently, Liu et al. [15] introduced an Improved Particle Swarm Optimization (IPSO) algorithm for the fixture layout optimization of large thin-walled parts.

Learning-based methods have recently emerged as an alternative strategy. Selvakumar et al. [16] combined Artificial Neural Networks (ANNs) with Design of Experiments (DOE) for machining fixture layout optimization. Wang et al. [17] integrated neural networks with the Bat Algorithm (BA) to minimize workpiece deformation. Low et al. [18] proposed a Reinforcement Learning (RL) framework for fixture design, while Wang et al. [19] presented a physics-guided RL approach in which the agent is trained through interaction with an FEA-based simulation.

Although the problem of fixture layout optimization has been extensively investigated, most existing work focuses on applications in the *metal industry*, where the objective is to determine the number and placement of fixtures according to the “N-2-1” locating principle in order to minimize workpiece deformation. The “N-2-1” principle is a generalization of the classical “3-2-1” locating scheme. In the latter, three locators constrain the primary datum plane, two constrain the secondary datum, and one constrains the tertiary datum, thereby fully restricting the six rigid-body degrees of freedom of the workpiece. The “N-2-1” principle replaces the three primary locators with N locators, while retaining two and one locators on the secondary and tertiary surfaces [17].

This framework cannot be directly applied to work centers in the *wood industry*, where the workpiece can only be supported from below. Furthermore, the fixtures adopted in the metal industry generally differ from those used in the wood industry, as they typically do not allow rotation. While allowing fixture rotation can significantly improve workpiece stability, it also substantially increases the number of possible solutions and therefore the complexity of the optimization problem.

3. Background

In this section, we introduce the concept of principal moments of inertia. In the proposed resolution strategies, the moments of inertia are used to define the objective function of the fixture layout optimization problem.

3.1. Principal moments of inertia

To maximize the stability of the workpiece through fixture placement, the *principal moments* of inertia of the fixture layout system can be analyzed.

The moment of inertia is a geometric property that quantifies a structure’s resistance to bending and torsional forces. It is computed from the distribution of mass relative to a given axis and plays a crucial role in determining the stiffness of structural elements under load. Higher moments of inertia indicate greater resistance to deformation, making this quantity a key factor in the design of fixtures that minimize workpiece deflection during machining operations [20, 21].

To compute the principal moments of inertia of a rectangular fixture system [22, 23, 24], each fixture can be represented as a polygon i of m vertices $(x_{i,1}, y_{i,1}), \dots, (x_{i,m}, y_{i,m})$ with area A_i , centroid coordinates (cx_i, cy_i) and rotation angle α_i . We first compute the *absolute moments* of inertia of each polygon with respect to the global

coordinate system:

$$\begin{aligned} J'_{x,i} &= \frac{1}{12} \sum_{j=1}^m (y_{i,j}^2 + y_{i,j}y_{i,j+1} + y_{i,j+1}^2)(x_{i,j}y_{i,j+1} - x_{i,j+1}y_{i,j}) \\ J'_{y,i} &= \frac{1}{12} \sum_{j=1}^m (x_{i,j}^2 + x_{i,j}x_{i,j+1} + x_{i,j+1}^2)(x_{i,j}y_{i,j+1} - x_{i,j+1}y_{i,j}) \\ J'_{xy,i} &= \frac{1}{24} \sum_{j=1}^m (x_{i,j}y_{i,j+1} + 2x_{i,j}y_{i,j} + 2x_{i,j+1}y_{i,j+1} + x_{i,j+1}y_{i,j})(x_{i,j}y_{i,j+1} - x_{i,j+1}y_{i,j}). \end{aligned}$$

The moments of inertia referred to the centroid of each polygon are then computed as:

$$J''_{x,i} = J'_{x,i} - A_i cy_i^2, \quad J''_{y,i} = J'_{y,i} - A_i cx_i^2, \quad J''_{xy,i} = J'_{xy,i} - A_i cx_i cy_i.$$

Since a polygon can be rotated by an angle α_i , the corresponding rotated moments are:

$$\begin{aligned} J_{x,i} &= J''_{x,i} \cos^2 \alpha_i + J''_{y,i} \sin^2 \alpha_i - 2J''_{xy,i} \sin \alpha_i \cos \alpha_i, \\ J_{y,i} &= J''_{x,i} \sin^2 \alpha_i + J''_{y,i} \cos^2 \alpha_i - 2J''_{xy,i} \sin \alpha_i \cos \alpha_i, \\ J_{xy,i} &= (J''_{x,i} - J''_{y,i}) \sin \alpha_i \cos \alpha_i + J''_{xy,i} (\cos^2 \alpha_i - \sin^2 \alpha_i). \end{aligned}$$

The total moments of inertia with respect to the global axes are then computed as:

$$\begin{aligned} J_x &= \sum_{i=1}^n J_{x,i} + A_i (cy_i - g_y)^2, & J_y &= \sum_{i=1}^n J_{y,i} + A_i (cx_i - g_x)^2, \\ J_{xy} &= \sum_{i=1}^n J_{xy,i} + A_i (cx_i - g_x)(cy_i - g_y) \end{aligned}$$

where g_x and g_y denote the coordinates of the overall centroid of the system, computed as:

$$g_x = \frac{\sum_{i=1}^n A_i cx_i}{A}, \quad g_y = \frac{\sum_{i=1}^n A_i cy_i}{A}$$

where $A = \sum_{i=1}^n A_i$ is the total area.

The principal moments of inertia of the overall system are obtained by first evaluating:

$$J'_{xg} = J_x - A g_y^2, \quad J'_{yg} = J_y - A g_x^2, \quad J'_{xyg} = J_{xy} - A g_x g_y,$$

and then computing:

$$\begin{aligned} J_\xi &= \frac{1}{2} \left((J'_{xg} + J'_{yg}) - \sqrt{(J'_{xg} - J'_{yg})^2 + 4(J'_{xyg})^2} \right), \\ J_\eta &= \frac{1}{2} \left((J'_{xg} + J'_{yg}) + \sqrt{(J'_{xg} - J'_{yg})^2 + 4(J'_{xyg})^2} \right). \end{aligned}$$

Layouts that provide good stability for the workpiece are those maximizing the sum:

$$f_{iner} = J_\xi + J_\eta \quad (1)$$

4. Problem description

Given the workpiece geometry and the available fixture types and quantities, the objective is to determine which fixtures should be selected and their placement so as to maximize workpiece stability during machining.

To ensure the feasibility of a placement, the following constraints must be satisfied:

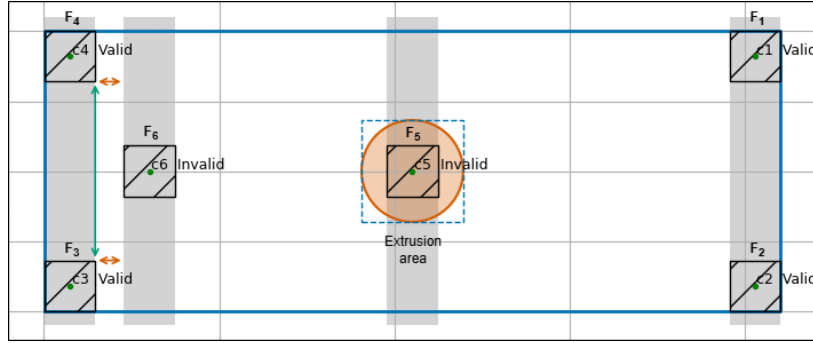


Figure 2. Example of valid and invalid fixture placements. For a fixture placement to be valid, the fixture must lie within the workpiece boundaries, satisfy the required safety distances from the other fixtures, and not be positioned beneath an area to be extruded.

1. the number of fixtures placed for each type must not exceed the number available;
2. fixtures must be placed so as to satisfy the specified minimum horizontal and vertical safety distances;
3. fixtures must be positioned within the workpiece boundaries and beneath its surface;
4. fixtures must not overlap each other;
5. fixtures must not be located beneath areas to be extruded, to prevent damage.

To illustrate the constraints, Figure 2 shows examples of valid and invalid fixture placements. Fixture F_6 is invalid because it violates the minimum horizontal safety distance with respect to fixtures F_3 and F_4 , whereas fixture F_5 is invalid because it is positioned beneath an area to be extruded.

The formulation of these constraints depends on the chosen solution strategy and the corresponding decision variables and is therefore presented separately for each of the considered solution techniques. All approaches, however, are defined over the same set of input parameters listed below:

- the width W_{Tab} and height H_{Tab} of the machine worktable;
- the minimum horizontal distance W_{Min} between two fixtures placed on different bars;
- the minimum vertical distance H_{Min} between two fixtures placed on the same bar;
- the number N_{Types} of different fixture types;
- the number N_{Fix}_i of available fixtures for each type $i \in [1..N_{Types}]$;
- the total number of available fixtures $N_{Fix} = \sum_{i=1}^{N_{Types}} N_{Fix}_i$;
- the minimum N_{Min} and maximum N_{Max} number of fixtures to place;
- the width W_{Fix}_i and height H_{Fix}_i of each fixture of type i ;
- the workpiece shape, represented by a set of linear inequalities of the form $ax + by + c \leq 0$;
- the number N_{Ext} of areas to be extruded;
- the width W_{Ext}_i , height H_{Ext}_i , and the coordinates (X_{Ext}_i, Y_{Ext}_i) of the lower-left corner of the smallest axis-aligned rectangle enclosing each extrusion area $i \in [1..N_{Ext}]$.

Extrusion areas are typically represented by simple geometric shapes, such as circles, rectangles, or ellipses, whereas more complex geometries are refined manually. To facilitate the solution process, each extrusion area is approximated by its smallest enclosing rectangle (as illustrated in Figure 2), which is represented by the coordinates of its lower-left corner together with its width and height.

To evaluate the quality of a fixture layout, two objective functions are considered. The CP and MIP models optimize the objective function f_{dist} proposed in [3], which maximizes the sum of the pairwise Manhattan distances between fixture centroids, augmented by the total area of the selected fixtures:

$$f_{dist} = \sum_{1 \leq i < j \leq n} (|cx_i - cx_j| + |cy_i - cy_j|) + \sum_{k=1}^n (WFix_{t_k} \cdot HFix_{t_k}) \quad (2)$$

where (x_k, y_k) are the coordinates of the lower-left corner of the k -th placed fixture, having type t_k (n is the total number of placed fixtures), and centroid $(cx_k, cy_k) = (x_k + \frac{WFix_{t_k}}{2}, y_k + \frac{HFix_{t_k}}{2})$.

The use of the f_{dist} objective function for the CP and MIP models is motivated by the fact that computing the principal moment of inertia expressed in Eq. 1 involves nonlinear operations, including squaring, square roots, and transcendental functions (sine and cosine), which substantially increase the computational effort required by CP and MIP solvers [25].

The Q-learning algorithm, on the other hand, is not affected by the nonlinear computations required to evaluate f_{iner} and therefore directly incorporates the moments of inertia into its reward function, as described in Section 7.

The CP model, the MIP model, and the Q-learning algorithm are used to generate initial feasible layouts. For each instance, the best layout produced by these methods is then refined using the PSO algorithm described in Section 8, which optimizes f_{iner} while introducing the fixture rotation angle as an additional decision variable.

5. CP model

The CP model extends the formulation presented in [3] through a revised definition of the decision variables and a reformulation of some constraints, resulting in improved resolution times on the tested instances. The model is defined over the parameters introduced in Section 4 and maximizes the objective function f_{dist} defined in Equation 2.

5.1. Variables

The fixture placement and type are represented by three arrays of integer decision variables, x , y , and t , for $i = 1, \dots, NMax$:

- $(x_i, y_i) \in [0..WTab] \times [0..HTab]$ denotes the coordinates of the lower-left corner of the i -th fixture;
- $t_i \in [0..NTypes]$ denotes the type of the i -th fixture, where type 0 represents an unused fixture.

For each fixture type $i \in [1..NTypes]$, an integer variable $n_i \in [0..NFix_i]$ denotes the number of selected fixtures of type i , and a variable $n \in [NMin..NMax]$ represents the total number of selected fixtures.

Compared with the formulation presented in [3], the arrays x and y and the variable n are now bounded by $NMax$, rather than by $NFix$, since the maximum number of fixtures that can be placed is known for each workpiece and $NMax \leq NFix$, this tighter bound reduces the search space.

5.2. Constraints

For each fixture type $i \in [1..NTypes]$, the count global constraint is used to compute the number of occurrences of value i in the array t :

$$n_i = \text{count}(t, i).$$

The total number of selected fixtures is then defined as

$$n = \sum_{i=1}^{NTypes} n_i.$$

Fixture availability is enforced by requiring that the number of selected fixtures of each type does not exceed the number available:

$$n_i \leq \text{NFix}_i, \quad \forall i \in [1..\text{NTypes}].$$

Since not all available fixtures must necessarily be used, we enforce the following invariant:

$$(i > n) \iff (x_i = \text{WTab} \wedge y_i = \text{HTab} \wedge t_i = 0).$$

This invariant ensures that all unused fixtures are assigned the dummy type 0 and placed at the dummy location (WTab, HTab), which lies outside the feasible region.

We guarantee that exactly $\text{NMax} - n$ fixtures remain unused by imposing

$$\text{count}(t, 0) = \text{NMax} - n.$$

The workpiece shape is defined by linear inequalities of the form $a x + b y + c \leq 0$. To ensure that each selected fixture i is placed within the workpiece, we must satisfy $a x'_i + b y'_i + c \leq 0$, where:

$$x'_i = \begin{cases} x_i & \text{if } a \leq 0, \\ x_i + \text{WFix}_{t_i} & \text{if } a > 0, \end{cases} \quad y'_i = \begin{cases} y_i & \text{if } b \leq 0, \\ y_i + \text{HFix}_{t_i} & \text{if } b > 0. \end{cases}$$

Areas to be extruded are approximated by axis-aligned rectangles. The non-overlap between these areas and the fixtures is enforced using the `diffn` global constraint [26]. Given a set of rectangles specified by their lower-left corner coordinates and dimensions, the `diffn` constraint ensures that no two rectangles overlap:

$$\text{diffn}(x \oplus \text{XExt}, y \oplus \text{YExt}, [\text{WFix}_{t_i} \mid i \in [1..n]] \oplus \text{WExt}, [\text{HFix}_{t_i} \mid i \in [1..n]] \oplus \text{HExt})$$

where $\oplus$ denotes array concatenation. Specifically, we use the non-strict version of `diffn`, in which zero-width rectangles can be positioned anywhere, ignoring the $\text{NMax} - n$ unused fixtures.

To enforce the minimum *vertical* distance HMin between fixtures placed on the same bar, and the minimum *horizontal* distance WMin between fixtures placed on different bars we impose:

$$\forall 1 \leq i < j \leq n : \begin{cases} y_i + \text{HFix}_{t_i} + \text{HMin} \leq y_j \vee y_j + \text{HFix}_{t_j} + \text{HMin} \leq y_i, & \text{if } cx_i = cx_j, \\ x_i + \text{WFix}_{t_i} + \text{WMin} \leq x_j, & \text{if } cx_i \neq cx_j. \end{cases}$$

where cx_i denotes the x -coordinate of the centroid of fixture i . Centroid coordinates are used instead of lower-left coordinates because fixtures may be wider than their supporting bar: while x_i may fall outside the horizontal span of the bar, the centroid cx_i always lies within it.

To compute the fixture centroid coordinates while maintaining integer variables, they are computed using integer division of the fixture dimensions, for each fixture $i \in [1..\text{NMax}]$, we define:

$$cx_i = x_i + \left\lfloor \frac{\text{WFix}_{t_i}}{2} \right\rfloor, \quad cy_i = y_i + \left\lfloor \frac{\text{HFix}_{t_i}}{2} \right\rfloor. \quad (3)$$

To improve the propagation of the integer division, the following redundant linear constraints were introduced:

$$\begin{aligned} 2 \cdot cx_i - 2 \cdot x_i &= \text{WFix}_{t_i} - (\text{WFix}_{t_i} \bmod 2), \\ 2 \cdot cy_i - 2 \cdot y_i &= \text{HFix}_{t_i} - (\text{HFix}_{t_i} \bmod 2). \end{aligned}$$

Finally, the model exhibits symmetries whenever fixtures of the same type are permuted without changing their spatial arrangement. To eliminate these equivalent solutions, a lexicographic ordering is imposed on the fixture positions using the `lex_chain` global constraint:

$$(x_{i-1}, y_{i-1}) \leq (x_i, y_i), \quad i \in [2..\text{NMax}].$$

6. MIP model

In this section, we present the Mixed-Integer Programming (MIP) formulation for the fixture layout optimization problem. Unlike the CP model, the MIP formulation does not rely on global constraints. Instead, it represents the problem as a system of linear constraints over continuous and binary decision variables. This relaxation comes from the nature of MIP solving which, unlike most CP solvers, naturally handles non-negative continuous variables.

6.1. Variables

Variables x_i and y_i , for $i = 1, \dots, \text{NMax}$, have the same semantics as those defined in Section 5.1, with the only difference that they are continuous rather than discrete.

Fixture types are encoded by a binary matrix T of size $\text{NMax} \times \text{NTypes}$, where

$$T_{i,j} = \begin{cases} 1 & \text{if fixture } i \text{ is selected and has type } j, \\ 0 & \text{otherwise.} \end{cases}$$

To represent the width, height, and type of each fixture, auxiliary variables w_i , h_i , and t_i are introduced for each $i \in [1 \dots \text{NMax}]$.

To linearize the constraints presented in Section 5.2, the following auxiliary binary variables are defined:

- $used_i$, which indicates whether fixture i is selected;
- $same_bar_{i,j}$, which indicates whether fixtures i and j are placed on the same bar;
- $above_in_bar_{i,j}$ and $below_in_bar_{i,j}$, which indicate whether fixture i is positioned above or below fixture j on the same bar;
- $above_{i,j}$, $below_{i,j}$, $left_{i,j}$, and $right_{i,j}$, which indicate whether fixture i is positioned above, below, to the left, or to the right of fixture j , respectively;
- $above_ext_{i,j}$, $below_ext_{i,j}$, $left_ext_{i,j}$, and $right_ext_{i,j}$, which indicate whether fixture i is positioned above, below, to the left, or to the right of extrusion area j , respectively.

6.2. Constraints

For each fixture $i \in [1..\text{NMax}]$, the following equations are imposed:

$$used_i = \sum_{j=1}^{\text{NTypes}} T_{i,j}, \quad w_i = \sum_{j=1}^{\text{NTypes}} \text{WFix}_j \cdot T_{i,j}, \quad h_i = \sum_{j=1}^{\text{NTypes}} \text{HFix}_j \cdot T_{i,j}, \quad t_i = \sum_{j=1}^{\text{NTypes}} j \cdot T_{i,j}.$$

These equations ensure that the width, height, and type assigned to each fixture are consistent and that unused fixtures have dimensions and type equal to 0.

To respect fixtures availability constraints, we impose:

$$\sum_{i=1}^{\text{NMax}} T_{i,j} \leq \text{NFix}_j, \quad \forall j \in [1..\text{NTypes}] \quad \text{NMin} \leq \sum_{i=1}^{\text{NMax}} used_i \leq \text{NMax}$$

To determine whether two fixtures lie on the same bar for all $i \in [1..\text{NMax}]$ and $j \in [i + 1..\text{NMax}]$ we impose the following constraints:

$$\begin{aligned} same_bar_{i,j} &\leq used_i & same_bar_{i,j} &\leq used_j \\ same_bar_{i,j} &= above_in_bar_{i,j} + below_in_bar_{i,j} \\ cx_i - cx_j &\leq M(1 - same_bar_{i,j}) & cx_j - cx_i &\leq M(1 - same_bar_{i,j}) \end{aligned}$$

where $M = WTab \cdot HTab$ is the “big- M ” constant. Then to enforce the appropriate safety distances we use the following inequalities:

$$\begin{aligned} cx_i + WMin &\leq cx_j + M(same_bar_{i,j} + right_{i,j}) \\ cx_j + WMin &\leq cx_i + M(same_bar_{i,j} + left_{i,j}) \\ y_i + h_i + HMin - y_j &\leq M(1 - above_in_bar_{i,j}) \\ y_j + h_j + HMin - y_i &\leq M(1 - below_in_bar_{i,j}). \end{aligned}$$

To prevent overlap between fixtures, for all $i, j \in [1..NMax]$ with $i < j$, we impose:

$$\begin{aligned} above_{i,j} + below_{i,j} + left_{i,j} + right_{i,j} &\geq 1 \\ x_i + w_i &\leq x_j + M(1 - left_{i,j}) & y_i + h_i &\leq y_j + M(1 - below_{i,j}) \\ x_j + w_j &\leq x_i + M(1 - right_{i,j}) & y_j + h_j &\leq y_i + M(1 - above_{i,j}) \end{aligned}$$

Analogous constraints are introduced to prevent overlap between fixtures and extrusion areas. For each fixture $i \in [1..NMax]$ and extrusion area $j \in [1..NExt]$, the same inequalities are applied using the binary variables $above_ext_{i,j}$, $below_ext_{i,j}$, $left_ext_{i,j}$, and $right_ext_{i,j}$.

The workpiece is defined by linear inequalities of the form $ax + by + c \leq 0$, so for each fixture $i \in [1..NMax]$ we impose a $x'_i + by'_i + c \leq 0$, where x'_i and y'_i are defined as in the CP model (see Section 5.2).

To break symmetries, we enforce an ordering on fixture types: $t_i \geq t_{i+1}$ for $i \in [1..NMax - 1]$.

Finally, the absolute value terms appearing in f_{dist} are linearized using a big- M formulation, unfolding each term of the form $u = |v|$ into:

$$u \geq v \wedge u \geq -v \wedge u \leq v + M(1 - b) \wedge u \leq -v + Mb \wedge b \in \{0, 1\}.$$

7. Q-learning algorithm

We compared the solutions obtained using the CP and MIP models with those produced by a Q-learning based algorithm.

In Q-learning, an agent learns through repeated interactions with the environment by estimating the action-value function $Q(s, a)$, which represents the expected *cumulative discounted reward* obtained by executing action a in state s and subsequently following the optimal policy [27].

After each interaction, the action-value function is updated according to the following rule [28]:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right], \quad (4)$$

where α is the learning rate, which determines the influence of newly acquired information on the current estimate, and γ is the discount factor, which balances the importance of immediate and future rewards. In Equation (4), the quantity inside the square brackets is the *temporal-difference (TD) error*, which measures the difference between the current estimate and the updated estimate obtained after observing the transition from state s_t to state s_{t+1} . The term r_{t+1} denotes the reward obtained after executing action a_t , while $\max_{a_{t+1}} Q(s_{t+1}, a_{t+1})$ represents the estimated value of the best action available in the successor state.

In our formulation, the environment corresponds to the considered workpiece. At each decision step, the state of the environment is defined as

$$s_t = (n_t, fix_t),$$

where n_t is the number of fixtures already placed at step t , while $fix_t = [fix_t^1, fix_t^2, \dots, fix_t^{NTypes}]$ is an integer array of size $NTypes$, whose components denote the remaining availability of the different fixture types.

An action consists of selecting a fixture type together with a feasible centroid position,

$$a_t = (type_t, (cx_t, cy_t)).$$

Algorithm 1 Q-learning for Fixture Layout Optimization

Require: Number of episodes E , maximum number of steps per episode T , learning rate α , discount factor γ , exploration rate ϵ , exploration decay factor ρ

Ensure: Best feasible layout $\mathcal{L}^*$

- 1: Compute all candidate centroid positions
- 2: Remove centroid positions violating the constraints and set $\mathcal{A}(s_1)$
- 3: Initialize $Q(s, a) = 0$
- 4: $\mathcal{L}^* \leftarrow \emptyset$
- 5: **for** $ep = 1$ to E **do**
- 6: **for** $t = 1$ to T **do**
- 7: **if** $\mathcal{A}(s_t) = \emptyset$ **then**
- 8: **break**
- 9: **end if**
- 10: **if** random number $< \epsilon$ **then** ▷ exploration
- 11: Select a random feasible action $a_t \in \mathcal{A}(s_t)$
- 12: **else** ▷ exploitation
- 13: Select $a_t = \arg \max_{a \in \mathcal{A}(s_t)} Q(s_t, a)$
- 14: **end if**
- 15: Perform action a_t by placing the selected fixture
- 16: Recompute the feasible action set $\mathcal{A}(s_{t+1})$
- 17: Compute reward r_t according to Eq. 5
- 18: Observe successor state s_{t+1}
- 19: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$
- 20: $s_t \leftarrow s_{t+1}$
- 21: Update $\mathcal{L}_{ep}$ with a_t
- 22: **end for**
- 23: **if** $\mathcal{L}_{ep}$ is better than $\mathcal{L}^*$ **then**
- 24: $\mathcal{L}^* \leftarrow \mathcal{L}_{ep}$
- 25: **end if**
- 26: $\epsilon \leftarrow \rho \epsilon$
- 27: Reset the environment to the initial state s_1
- 28: Reset the action space to $\mathcal{A}(s_1)$
- 29: **end for**
- 30: Evaluate the learned policy using greedy action selection
- 31: **return** $\mathcal{L}^*$

The fixture centroid is used as the reference point for positioning instead of the lower-left corner adopted in the CP and MIP models, since it simplifies both the enforcement of the horizontal and vertical safety-distance constraints, as described in Section 5.2, and the identification of feasible actions.

The action to be executed is selected according to an ϵ -greedy policy [29]. At each decision step, the agent selects the action with the highest estimated Q -value with probability $1 - \epsilon$, whereas with probability ϵ it selects a random action, balancing exploration and exploitation of the environment.

The Q-learning based procedure adopted for fixture layout optimization is described in Algorithm 1. The algorithm takes as inputs the number of training episodes E , the maximum number of steps per episode T , the learning rate α , the discount factor γ , the exploration rate ϵ , and the exploration decay factor ρ . The output is the best feasible fixture layout $\mathcal{L}^*$ obtained by applying the learned policy.

Given the fixture dimensions, it is known *a priori* that some positions within the workpiece cannot accommodate a fixture, particularly those near the workpiece boundary or close to areas to be extruded. During the initialization phase, in lines 1–4 of Algorithm 1, centroid positions that would result in fixtures extending outside the workpiece boundaries or overlapping extrusion areas are removed from the set of possible actions $\mathcal{A}(s_t)$, considering the dimensions of the smallest fixture as reference.

Figure 3 shows an example of the set of feasible actions for a given workpiece, considering two available fixture types: square and rectangular. Green dots indicate centroid positions where both fixture types can be placed, whereas red dots indicate positions where only rectangular fixtures can be placed.

The algorithm iterates over the training episodes and the corresponding steps. An episode terminates when no feasible actions remain or the maximum number of steps is reached, as shown in lines 5–9 of Algorithm 1.

The agent selects actions according to the ϵ -greedy policy (lines 10–15 of Algorithm 1), balancing the exploration

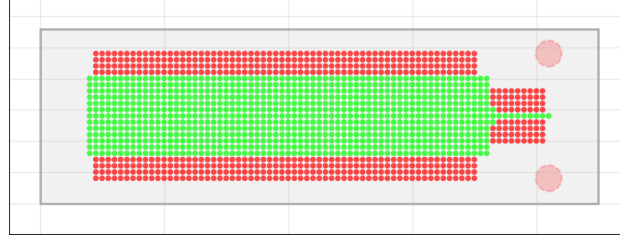


Figure 3. Example of the available actions for a given workpiece. Green dots represent centroid positions where both square and rectangular fixtures can be placed, whereas red dots represent positions where only rectangular fixtures can be placed. The workpiece area is highlighted in light gray, while the areas to be extruded are highlighted in red.

of new placements with the exploitation of previously learned actions.

Each action selected is evaluated through the reward associated with the transition from s_t to s_{t+1} . The reward function combines the improvement obtained in the moment of inertia with additional terms promoting layouts composed of a larger number of fixtures, larger fixture dimensions, and increased spatial separation between fixtures:

$$r_{t+1} = \lambda_{iner} \Delta f_{iner} + \lambda_n n_{t+1}^2 + \lambda_d \bar{d} + \begin{cases} b_{NMin}, & \text{if } n_{t+1} \geq NMin, \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

In the equation:

- Δf_{iner} is the variation of f_{iner} between the previous and current states;
- n_{t+1} is the number of fixtures in the resulting layout. The quadratic term promotes layouts with a larger number of fixtures;
- $\bar{d}$ is the average Euclidean distance between the newly placed fixture and the previously placed fixtures, promoting layouts with a larger spatial separation;
- λ_{iner} , λ_n , and λ_d are weighting coefficients balancing the contribution of the different reward components;
- b_{NMin} is an additional feasibility bonus assigned when the minimum required number of fixtures, $NMin$, has been placed.

After each fixture placement, all candidate centroid positions that no longer satisfy the safety-distance constraints are removed from the feasible action set, as shown in line 16 of Algorithm 1. Consequently, the action set progressively shrinks as the layout is constructed, ensuring that all geometric constraints are satisfied.

The only constraint that cannot be enforced by restricting the feasible action set is the placement of the minimum number of required fixtures $NMin$. For this reason, the feasibility bonus b_{NMin} is included in the reward function to encourage the agent to generate layouts satisfying this constraint.

After executing the selected action, the corresponding reward is computed, the Q -function value associated with state s_t and action a_t is evaluated and the learned policy is updated (lines 17–21 of Algorithm 1).

Finally, after all training episodes have been completed, the learned policy is evaluated to obtain the fixture layout for the considered workpiece, as shown in lines 30–31 of Algorithm 1.

8. PSO algorithm

Particle Swarm Optimization (PSO) is employed as a meta-heuristic refinement procedure, using the solutions obtained from the CP model, the MIP models and the Q-learning-based approach, as starting layouts.

PSO is an iterative optimization technique inspired by swarm intelligence and originally motivated by studies of the collective behavior of bird flocks [30, 31]. The meta-heuristic operates on a swarm of particles, each characterized by a position vector p_i and a velocity vector v_i . The position of each particle represents a candidate solution and is

associated with a fitness value measuring its quality. At each iteration $t = 1, 2, \dots$, particles explore the search space by updating their velocities and positions according to the following equations:

$$\begin{aligned} v_i^{(t+1)} &= w v_i^{(t)} + c_1 r_1 (p_i^* - p_i^{(t)}) + c_2 r_2 (g^* - p_i^{(t)}) \\ p_i^{(t+1)} &= p_i^{(t)} + v_i^{(t+1)} \end{aligned}$$

where:

- w is the *inertia weight*, controlling how much of the previous velocity is retained;
- c_1 is the *cognitive coefficient*, guiding the particle toward its *personal best position* p_i^* ;
- c_2 is the *social coefficient*, guiding particles toward the *global best position* g^* of the swarm;
- r_1 and r_2 are random numbers uniformly distributed in $[0, 1]$, introducing stochasticity into the cognitive and social components, respectively.

In the proposed approach, PSO further improves the provided initial layouts by explicitly considering fixture rotation. The objective is to maximize the sum of the principal moments of inertia, i.e., the objective function $f_{iner} = J_\xi + J_\eta$ defined in Section 3.

In our formulation, the position of each particle $s \in [1, \text{swarm_size}]$ is represented by a matrix P_s with four rows and NMax columns, where NMax denotes the maximum number of available fixtures. Each column of P_s represents a fixture, while each row corresponds to one of the decision variables:

- $P_s[1, j]$: the x -coordinate of the lower-left corner of the j -th fixture;
- $P_s[2, j]$: the y -coordinate of the lower-left corner of the j -th fixture;
- $P_s[3, j]$: the type t associated with the j -th fixture;
- $P_s[4, j]$: the rotation angle α of the j -th fixture.

Each position matrix P_s represents a candidate solution associated with particle s . The availability of an initial solution obtained from the CP model, the MIP model, or the Q-learning algorithm, ensures that the swarm is initialized from feasible layouts.

Algorithm 2 presents the pseudocode of the implemented PSO approach. Given the swarm size `swarm_size`, the maximum number of iterations `max_iter`, the inertia weight w , the cognitive coefficient c_1 , the social coefficient c_2 , the variable bounds, the initial feasible layout P_0 , and the maximum number of fixtures NMax, the algorithm returns the optimized feasible fixture layout P^* .

During the initialization phase (lines 1–6 of Algorithm 2), the first particle is assigned the provided initial solution P_0 , whereas the remaining particles are initialized using randomly perturbed versions of P_0 . The velocity matrix V_s of each particle s , which stores the velocity components corresponding to each element of the position matrix P_s , is randomly initialized, and the personal best position P_s^* is initially set equal to the particle's starting position.

In this formulation, the linear constraints of the MIP model are incorporated into the PSO framework and treated as soft constraints, with the non-overlap constraints adapted to account for fixture rotations. Accordingly, for each particle position P_s , the violation of the j -th constraint is evaluated by means of a binary function $cv_j(P_s)$, which indicates whether the constraint is violated or satisfied. The overall constraint violation $cv(P_s)$ for position P_s is computed as the sum of each individual constraint violation.

Once all particles have been initialized, they are evaluated in terms of the objective function $f_{iner}(P_s)$ and the constraint violation $cv(P_s)$, as reported in lines 7–11 of Algorithm 2. Based on this evaluation, the global best position of the swarm, denoted by P^* , is identified.

When fixtures are axis-aligned, non-overlap can be enforced through coordinate-wise comparisons by exploiting the orientation of the rectangle edges. Introducing fixture rotations invalidates this criterion, since the global position of each vertex depends on the rotation angle and must be computed through trigonometric transformations.

Algorithm 2 Particle Swarm Optimization for Fixture Layout Optimization

Require: Swarm size `swarm_size`, maximum number of iterations `max_iter`, inertia weight w , cognitive coefficient c_1 , social coefficient c_2 , variable bounds, initial solution P_0 , maximum number of fixtures `NMax`

Ensure: Feasible solution P^* not worse than P_0

```

1: for  $s = 1$  to swarm_size do
2:   if  $s = 1$  then
3:      $P^* \leftarrow P_s \leftarrow P_0$ 
4:   else
5:      $P_s^* \leftarrow P_s \leftarrow P_0 + \text{random perturbation}$ 
6:   end if
7:   if  $cv(P_s) = 0$  then
8:     if  $f_{iner}(P_s) > f_{iner}(P^*)$  then
9:        $P^* \leftarrow P_s$ 
10:    end if
11:  end if
12:  Randomly initialize velocity matrix  $V_s$ 
13: end for
14: for  $k = 1$  to max_iter do
15:   for  $s = 1$  to swarm_size do
16:    for  $i = 1$  to 4 do
17:      for  $j = 1$  to NMax do                                ▷ Update velocity and position
18:        Randomly generate  $r_1, r_2 \in [0, 1]$ 
19:         $V_s[i, j] \leftarrow wV_s[i, j] + c_1r_1(P_s^*[i, j] - P_s[i, j]) + c_2r_2(P^*[i, j] - P_s[i, j])$ 
20:         $P_s[i, j] \leftarrow P_s[i, j] + V_s[i, j]$ 
21:      end for
22:    end for
23:    for  $j = 1$  to NMax do
24:      if fixture  $j$  lies outside the workpiece then
25:        Identify the nearest workpiece vertex and the fixture vertex farthest from it
26:        Translate fixture  $j$  by the minimum displacement required to satisfy the boundary constraint
27:      end if
28:      if fixture  $j$  has type 0 then
29:        Set  $P_s[1, j] \leftarrow \text{WTab}$  and  $P_s[2, j] \leftarrow \text{HTab}$ 
30:      end if
31:    end for
32:    if  $cv(P_s) = 0$  and  $cv(P_s^*) = 0$  then                                ▷ Deb's rules
33:      if  $f_{iner}(P_s) > f_{iner}(P_s^*)$  then
34:         $P_s^* \leftarrow P_s$ 
35:      if  $f_{iner}(P_s^*) > f_{iner}(P^*)$  then
36:         $P^* \leftarrow P_s^*$ 
37:      end if
38:    end if
39:    else if  $cv(P_s) < cv(P_s^*)$  then
40:       $P_s^* \leftarrow P_s$ 
41:    end if
42:  end for
43: end for
44: return  $P^*$ 

```

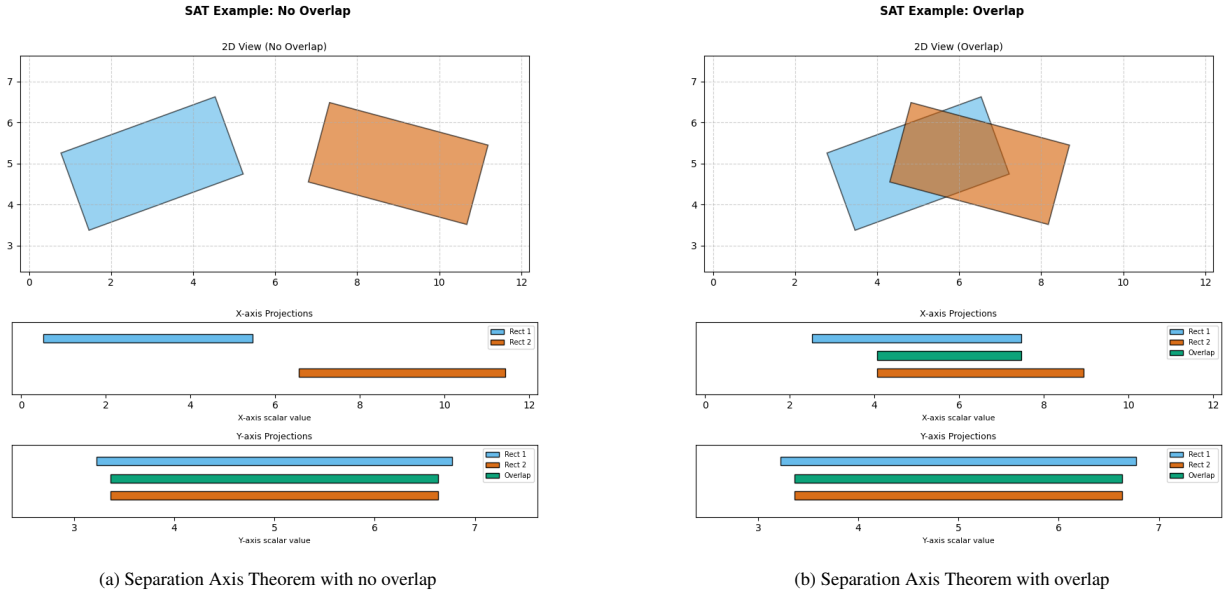


Figure 4. Separation Axis Theorem for oriented rectangles. Each figure consists of three subplots. The first subplot illustrates the spatial configuration of the rectangles. The second and third subplots show the projections of their principal axes onto the x - and y -axes, respectively. An overlap occurs when the projections intersect along both axes; the overlap is highlighted in black in the second and third subplots.

Consequently, non-overlap verification becomes orientation-dependent, increasing the implementation complexity. Furthermore, vertex-based checks are no longer sufficient for oriented rectangles, as two fixtures may intersect even when no vertex of either rectangle lies inside the other, for example in edge-to-edge configurations.

For this reason, the *Separation Axis Theorem* (SAT) [32] is adopted to enforce the non-overlap constraint. The theorem states that two oriented rectangles do not overlap if and only if there exists at least one axis along which their projections are disjoint. Such an axis is called a *separating axis*. Therefore, non-overlap is verified by projecting both rectangles onto the candidate separating axes and checking whether their projections are disjoint on at least one of them, as illustrated in Figure 4.

During the search phase (lines 18–20 of Algorithm 2), the velocity matrix V_s and the position matrix P_s associated with particle s are iteratively updated according to the standard PSO update equations.

Whenever possible, constraint violations arising from updated fixture positions are mitigated through dedicated geometric repair procedures, as reported in lines 24–26 of Algorithm 2. For each fixture, if its geometry lies partially or entirely outside the workpiece boundary, the nearest vertex of the workpiece polygon and the fixture vertex farthest from it are identified. The fixture is then translated by the minimum displacement required to satisfy the boundary constraint. The repair strategy is limited to geometric violations, as these constraints are the most suitable for correction without adversely affecting the satisfaction of other constraints.

In lines 28–30 of Algorithm 2 fixtures with type zero, representing unused fixtures, are assigned the dummy lower-left corner coordinates (WTab, HTab), as in the CP model formulation (see Section 5.2), and are excluded from constraint violation evaluations.

To guide the search toward constraint-respecting positions, as illustrated in lines 32–41 of Algorithm 2, candidate solutions are evaluated using Deb's rules [33]:

- Between two feasible solutions, retain the one with the higher objective value.
- Between a feasible and an infeasible solution, retain the feasible one.
- Between two infeasible solutions, retain the one with the lower constraint violation.

After updating the personal best position P_s^* of each particle s using Deb's rules, the global best position P^* of the swarm is updated accordingly. At the end of the final iteration, the best position found by the swarm is returned.

9. Evaluation

We evaluated the proposed resolution strategy on seven distinct workpieces: a stair step of a spiral staircase, a stair step of a simple staircase, a music speaker, a dashboard, a coffee table, a door, and a door with a porthole (see Table 3).

The work center considered was equipped with 24 square fixtures with dimensions 145×145 mm and 12 rectangular fixtures with dimensions 180×65 mm. The minimum and maximum numbers of fixtures were set to $N_{Min} = 3$ and $N_{Max} = 6$, respectively, for all workpieces except the coffee table and the two door workpieces, which, due to their larger dimensions, required $N_{Min} = 6$ and $N_{Max} = 10$, and $N_{Min} = 10$ and $N_{Max} = 20$, respectively.

Both the CP and MIP models were implemented in *MiniZinc* version 2.9.5 [34]. The CP model was solved using the CP solvers Gecode [35], Chuffed [36], and OR-Tools [37], whereas the MIP model was solved using the commercial solver Gurobi [38].

We also considered *Gecode + LNS*, a variant of Gecode implementing Large Neighborhood Search (LNS) [39], to solve the CP model, configuring it to explore the search space by randomly fixing 80% of the decision variables corresponding to the x - and y -coordinates, adopting a constant restart strategy triggered every 1000 explored nodes.

All solvers were executed in single-threaded mode to ensure a fair comparison. The experiments were performed on a 64-bit Windows machine equipped with an Intel Core i5 processor and 16 GB of RAM.

Table 1 reports the values obtained for the objective function f_{dist} for the different workpieces, together with the corresponding *a posteriori* values of f_{iner} , and the solve time t_{solve} required to reach the optimal solution.

Identical values of f_{dist} do not necessarily correspond to identical values of f_{iner} , as observed for the Dashboard workpiece. This is because the moments of inertia are highly sensitive to little displacement; therefore, if the solutions obtained by the solvers do not result in identical fixture placements, the value of f_{iner} may differ.

Among the CP solvers, Gecode delivers the best overall performance, finding feasible solutions for all the considered workpieces, whereas Chuffed and OR-Tools fail to find feasible solutions for the two door workpieces.

Gecode + LNS achieves the best results for the two door workpieces, producing layouts that outperform those obtained by Gurobi with the MIP model. For the remaining workpieces, however, Gurobi finds solutions that are

Table 1. Objective values for f_{dist} and f_{iner} , and solve time t_{solve} in seconds to reach the best f_{dist} solution. Timeout (300 s) is indicated by “–”. Solutions where the solver reached optimality are marked with *. For each workpiece, the maximum values of f_{dist} and f_{iner} are highlighted in bold, including ties.

Workpiece	Metric	Gecode	Gecode+LNS	Chuffed	OR-Tools	Gurobi
Stair step spiral staircase	f_{dist}	45655*	45655	45655*	45655*	45662.155*
	f_{iner}	2.13968×10^9	2.13968×10^9	2.13960×10^9	2.13968×10^9	2.17089×10^9
	t_{solve}	0.559	–	0.326	0.120	2.930
Stair step simple staircase	f_{dist}	72823	72823	72823*	72721	72826.000*
	f_{iner}	6.76573×10^9	6.76573×10^9	6.76530×10^9	6.48172×10^9	6.76638×10^9
	t_{solve}	–	–	3.386	–	0.220
Dashboard	f_{dist}	55582*	55582	55582*	55582*	55582.000*
	f_{iner}	7.10094×10^9	7.10094×10^9	7.08735×10^9	7.08735×10^9	7.08735×10^9
	t_{solve}	0.044	–	0.057	0.102	0.157
Speaker	f_{dist}	49548*	49548	49548*	49548*	49548.000*
	f_{iner}	2.93433×10^9	2.93433×10^9	2.93433×10^9	2.93433×10^9	2.93433×10^9
	t_{solve}	165.018	–	0.106	0.120	0.126
Coffee table	f_{dist}	137262	185351	192711	122920	201686.505
	f_{iner}	2.20750×10^{10}	2.19967×10^{10}	2.21449×10^{10}	9.10490×10^9	2.19472×10^{10}
	t_{solve}	–	–	–	–	–
Door	f_{dist}	261757	521632	–	–	355744.000
	f_{iner}	1.08442×10^{11}	1.52925×10^{11}	–	–	1.17690×10^{11}
	t_{solve}	–	–	–	–	–
Door with porthole	f_{dist}	261759	393343	–	–	298186.000
	f_{iner}	1.08422×10^{11}	1.52861×10^{11}	–	–	1.39714×10^{11}
	t_{solve}	–	–	–	–	–

Table 2. Objective values f_{iner} comparison: CP solvers (best among Gecode, LNS, Chuffed, OR-Tools), MIP solver (Gurobi), and RL approach. For each workpiece, the maximum value is highlighted in bold.

Workpiece	CP model	MIP model	Q-learning
Stair step spiral staircase	2.13968×10^9	2.17089×10^9	1.93822×10^9
Stair step simple staircase	6.76573×10^9	6.76638×10^9	5.65433×10^9
Dashboard	7.10094×10^9	7.08735×10^9	6.65446×10^9
Speaker	2.93433×10^9	2.93433×10^9	2.81248×10^9
Coffee table	2.29750×10^{10}	2.19472×10^{10}	1.56444×10^{10}
Door	1.52925×10^{11}	1.17690×10^{11}	1.14387×10^{11}
Door with porthole	1.52861×10^{11}	1.39714×10^{11}	1.45467×10^{11}

either superior or equivalent to those obtained by the CP solvers with respect to f_{dist} . This can be partly attributed to the use of continuous decision variables in the MIP model, which enlarges the feasible region and may therefore yield higher-quality solutions.

If we consider the information of the solve time presented in Table 1, among the CP solvers, with an imposed time limit of 300 s, Chuffed is the most robust, finding the optimal solution in four of the seven instances, and it is the only CP solver able to prove optimality for the stair step of the simple staircase. Gecode exhibits the greatest variability in performance, ranging from competitive runtimes on the easier instances to requiring 165.018 s for the speaker workpiece, while Chuffed and OR-Tools solve the same instance in only 0.106 s and 0.120 s, respectively.

Being a local-search-based approach, *Gecode* + *LNS* cannot prove optimality. Nevertheless, it reaches the same optimal value of f_{dist} as the other CP solvers for the simpler workpieces and achieves the best value of f_{iner} in four out of the seven considered workpieces, demonstrating the effectiveness of a local search strategy.

The Q-learning-based algorithm was implemented in Python 3.11.9 using the Gymnasium library version 1.3.0. The agent was trained on all the considered workpieces for 200 episodes, each consisting of 20 steps, using the following hyperparameters: learning rate $\alpha = 0.1$, discount factor $\gamma = 0.99$, initial exploration rate $\epsilon = 1.0$, and exploration decay factor $\rho = 0.995$.

Table 2 reports the results obtained for the different workpieces, comparing the corresponding values of the moment of inertia with those achieved by the best CP solver and the MIP model.

Overall, the Q-learning-based algorithm does not outperform either the CP or the MIP model. On average, the values of f_{iner} obtained by RL are 13% lower than those achieved by CP and 8% lower than those obtained with MIP. The only exception is the door with a porthole workpiece, for which Q-learning outperforms the solution returned by Gurobi for the MIP model.

For each workpiece, the best CP and MIP solutions in terms of f_{iner} , together with the Q-learning solution, were used as initial ones for the PSO algorithm.

To enable direct integration into the target industrial system, the PSO algorithm was implemented in Java 18.0.2. Its hyper-parameters were tuned using the Optuna framework [40], yielding the following configuration: `swarm_size` = 1374, `max_iter` = 2912, $w = 0.669$, $c_1 = 2.385$, and $c_2 = 0.558$.

Figure 5 shows the percentage improvement in f_{iner} with respect to the layout proposed by an expert operator. The first plot (Fig 5a) reports the improvements achieved by the CP model, the MIP model, and the Q-learning algorithm before PSO refinement, whereas the second plot (Fig ??) shows the corresponding results after PSO refinement, including those obtained by initializing PSO from the expert layout.

The largest workpieces, namely the coffee table, the door, and the door with porthole, represent the most challenging instances. Among the considered approaches, the CP model consistently achieves the best improvements over the expert layout and is the only one that improves upon the expert solution for the door workpiece.

The Q-learning algorithm outperforms the expert layout in four of the seven considered workpieces. The integration with PSO further improves the solutions generated by the Q-learning algorithm for the stair step of the spiral

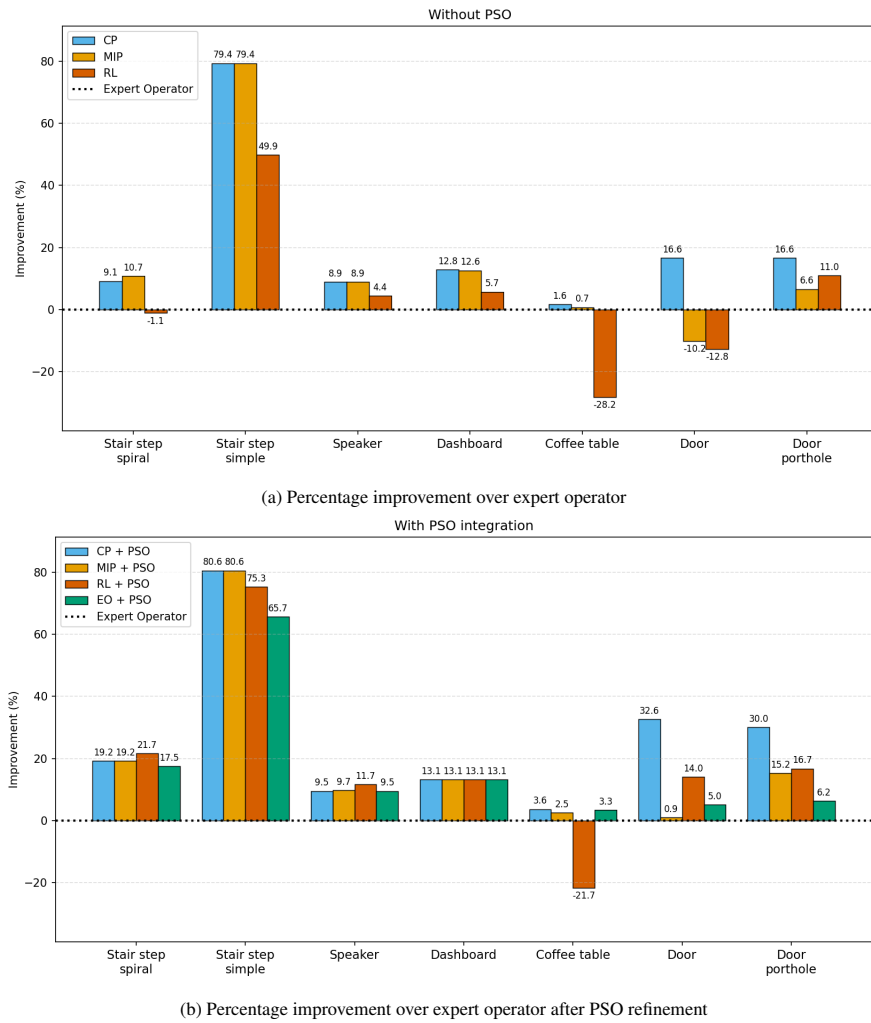


Figure 5. Percentage improvement relative to the solution provided by the expert operator. The first plot illustrates the improvements achieved by the CP model, MIP model and Q-learning algorithm without PSO integration, whereas the second plot presents the improvements obtained with PSO integration, alongside the improvement of the expert operator's solution when combined with PSO.

staircase and the door, allowing them to surpass the expert layout. Conversely, starting from the Q-learning solution for the coffee table, only marginal improvements are obtained, highlighting the importance of starting the PSO search from a high-quality initial solution.

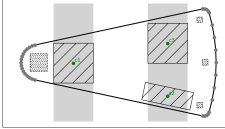
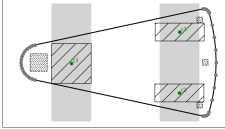
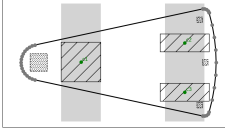
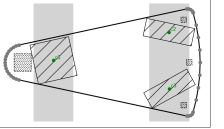
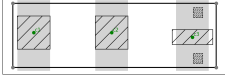
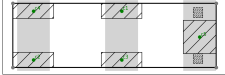
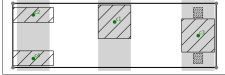
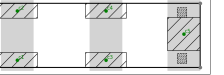
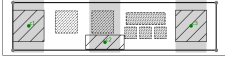
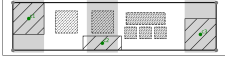
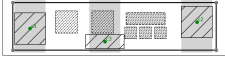
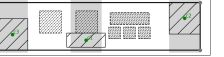
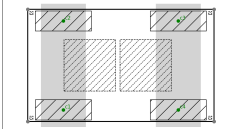
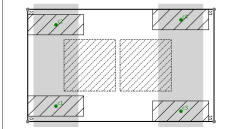
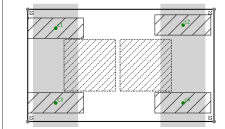
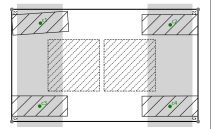
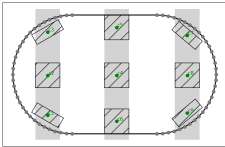
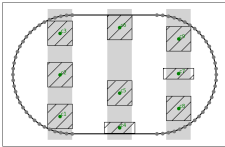
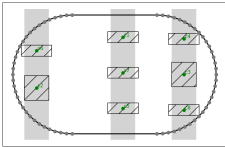
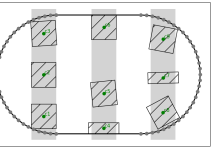
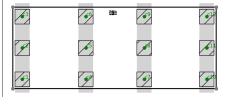
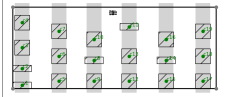
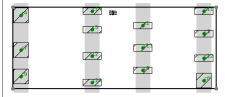
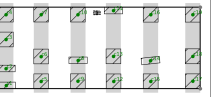
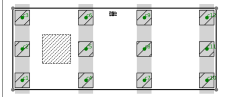
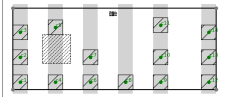
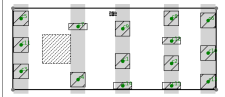
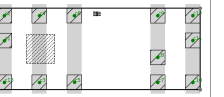
Table 3 reports visual comparisons of the best solutions obtained with the CP model, the MIP model, and the Q-learning algorithm for the different workpieces, together with the corresponding solutions refined by PSO. Note that, in the figures, the support bars can be positioned beneath the extrusion areas, since the fixture height prevents collisions between the cutting tools and the bars.

The layouts proposed by the Q-learning algorithm are qualitatively similar to those obtained with the CP and MIP models. An exception is the coffee table, where Q-learning tends to favour the placement of a larger number of rectangular fixtures instead of square ones, resulting in a lower value for the moment of inertia.

Rotation plays an important role in the stair step of the spiral staircase and coffee table workpieces, where fixture orientation improves adherence to the workpiece boundary. For the Speaker workpiece, the PSO refinement of the Q-learning solution introduces only a slight rotation of the upper-left fixture, yielding the marginal improvement reported in Figure 5.

The Stair step of the simple staircase exhibits the largest improvement over the expert layout. This is mainly

Table 3. Summary of the best solutions, corresponding solution providers, and moment of inertia values for the different workpieces. The workpiece perimeter is shown with solid lines. Dashed lines indicate extrusion areas, fixture regions are marked with diagonal hatching, and supporting bars are shown as shaded rectangles.

Workpiece	Expert Operator	Best CP/MIP	Q-learning	Best with PSO
Spiral stair step	1.96057×10^9 	MIP: 2.17089×10^9 	1.93822×10^9 	RL: 2.38520×10^9 
Simple stair step	3.77213×10^9 	MIP: 6.76638×10^9 	5.65433×10^9 	CP: 6.81108×10^9 
Dashboard	6.29544×10^9 	CP: 7.10094×10^9 	6.65446×10^9 	CP: 7.12226×10^9 
Speaker	2.69490×10^9 	CP, MIP: 2.93433×10^9 	2.81248×10^9 	RL: 3.00979×10^9 
Coffee table	2.17954×10^{10} 	CP: 2.21449×10^{10} 	1.56444×10^{10} 	CP: 2.25868×10^{10} 
Door	1.31109×10^{11} 	CP: 1.52925×10^{11} 	1.14387×10^{11} 	CP: 1.73826×10^{11} 
Door porthole	1.31109×10^{11} 	CP: 1.52861×10^{11} 	1.45467×10^{11} 	CP: 1.70507×10^{11} 

due to the use of a larger number of fixtures, five for the CP and MIP models and four for the Q-learning algorithm, compared with the three fixtures used by the expert operator.

For both the door and the door with a porthole workpieces, the expert operator adopts a symmetric fixture layout, whereas the PSO refinement tends to move the fixtures farther from the workpiece centre. Since the principal moments of inertia are maximised when the fixtures are placed away from the symmetry axes, this configuration yields higher values of f_{iner} . Moreover, in the door workpiece, the expert layout relies exclusively on square fixtures, whereas the optimization strategies increase the number of fixtures using even rectangular ones, thereby increasing the supported area of the workpiece.

Overall, the best trade-off between computational time and solution quality is achieved by integrating the CP model with PSO. Moreover, the CP formulation is expected to scale better to problems involving additional fixture types and multiple extrusion areas owing to the stronger propagation of the diffn global constraint and the fact that type and overlap decisions are modeled through discrete variables that cannot be relaxed, unlike the x and y variables in the MIP formulation.

10. Conclusion

This paper presented a comparative study of different solution strategies for the fixture layout optimization problem in wood industry.

The problem was first addressed through Constraint Programming (CP) and Mixed Integer Programming (MIP) models, whose performance was compared with that of a Q-learning-based algorithm. The experimental results showed that the CP formulation generally provides the best layouts in terms of the moment of inertia objective, with the *Gecode + LNS* solver consistently generating feasible solutions even for the most challenging instances, where other CP solvers fail.

Since the CP, MIP, and Q-learning approaches do not explicitly account for fixture rotations, their solutions were subsequently refined through a Particle Swarm Optimization (PSO) meta-heuristic, which treats fixture rotation as an additional decision variable, thereby addressing the complete optimization problem.

Among the considered approaches, the CP model refined by PSO provides the best trade-off between computational time and solution quality. Although the Q-learning algorithm does not always outperform the layouts proposed by the expert operator, its integration with PSO significantly improves the obtained solutions, surpassing the expert layouts for all the tested workpieces except the coffee table, for which the initial Q-learning solution was the weakest.

PSO also proves effective when initialized directly from the expert operator's layout, consistently improving the proposed solutions. This demonstrates its potential as a practical standalone refinement tool that can assist operators in further enhancing manually designed fixture layouts.

Future work may be focus on extending the proposed models to simultaneously optimize the fixture layouts of multiple workpieces on the machine worktable, as well as investigating alternative meta-heuristic and learning-based refinement strategies.

References

- [1] J. C. Trappey, C. R. Liu, A literature survey of fixture design automation, *The International Journal of Advanced Manufacturing Technology* 5 (3) (1990) 240–255.
- [2] M. R. Jackson, R. M. Parkin, N. Brown, Waves on wood, *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture* 216 (4) (2002) 475–497.
- [3] A. Vitali, R. Amadini, V. Maniezzo, M. Gabbrielli, Constraint solving and particle swarm optimization for fixture layout optimization, in: *32nd International Conference on Principles and Practice of Constraint Programming (CP 2026)*, Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2026, pp. 56–1.
- [4] S. Bhavikatti, *Finite element analysis*, New Age International, 2005.
- [5] R. Menassa, W. DeVries, Optimization methods applied to selecting support positions in fixture design.
- [6] W. Cai, S. J. Hu, J. Yuan, Deformable sheet metal fixturing: principles, algorithms, and simulations.
- [7] J. A. Camelio, S. J. Hu, D. Ceglarek, Impact of fixture design on sheet metal assembly variation, *Journal of Manufacturing Systems* 23 (3) (2004) 182–193.
- [8] B. Li, S. N. Melkote, Optimal fixture design accounting for the effect of workpiece dynamics, *The International Journal of Advanced Manufacturing Technology* 18 (10) (2001) 701–707.
- [9] B. Li, H. Yu, X. Yang, Y. Hu, Variation analysis and robust fixture design of a flexible fixturing system for sheet metal assembly (2010).
- [10] X. Zhang, W. Y. Yang, M. Li, Fixture layout and clamping force optimization for large-scale workpiece using augmented lagrangian method, *Applied Mechanics and Materials* 29 (2010) 560–565.
- [11] K. Kulankara, S. Satyanarayana, S. N. Melkote, Iterative fixture layout and clamping force optimization using the genetic algorithm, *J. Manuf. Sci. Eng.* 124 (1) (2002) 119–125.
- [12] Y. G. Liao, A genetic algorithm-based fixture locating positions and clamping schemes optimization, *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture* 217 (8) (2003) 1075–1083.
- [13] K. Padmanaban, K. Arulshri, G. Prabhakaran, Machining fixture layout design using ant colony algorithm based continuous optimization method, *The International Journal of Advanced Manufacturing Technology* 45 (9) (2009) 922–934.
- [14] K. Sundaraman, K. Padmanaban, M. Sabareeswaran, Optimization of machining fixture layout using integrated response surface methodology and evolutionary techniques, *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science* 230 (13) (2016) 2245–2259.
- [15] C. Liu, Y. Zheng, J. Wang, K. Jin, J. Yu, J. Liu, Fixture layout optimization for large thin-walled parts based on improved particle swarm optimization algorithm, *The International Journal of Advanced Manufacturing Technology* 132 (7) (2024) 3579–3592.
- [16] S. Selvakumar, K. Arulshri, K. Padmanaban, K. Sasikumar, Design and optimization of machining fixture layout using ann and doe, *The International Journal of Advanced Manufacturing Technology* 65 (9) (2013) 1573–1586.
- [17] Z. Wang, Y. Yang, B. Yang, Y. Kang, Optimal sheet metal fixture locating layout by combining radial basis function neural network and bat algorithm, *adv mech eng* 8 (12): 168781401668190 (2016).
- [18] D. W. Low, D. W. K. Neo, A. S. Kumar, A study on automatic fixture design using reinforcement learning, *The International Journal of Advanced Manufacturing Technology* 107 (5) (2020) 2303–2311.

- [19] Y. Wang, T. Lutz, X. Yue, J. Du, Smartfixture: Physics-guided reinforcement learning for automatic fixture layout design in manufacturing systems, *IIE Transactions* 57 (11) (2025) 1360–1375.
- [20] I. Kalkan, Deflection prediction for reinforced concrete beams through different effective moment of inertia expressions, *International Journal of Engineering Research and Development* 5 (1) (2013) 1–1.
- [21] P. H. Bischoff, S. P. Gross, Equivalent moment of inertia based on integration of curvature, *Journal of Composites for Construction* 15 (3) (2011) 263–273.
- [22] O. Belluzzi, *Scienza delle costruzioni*, Vol. 2, N. Zanichelli, 1961.
- [23] H. W. Coultas, *Theory of Structures: A Textbook Covering the Syllabuses of the B. Sc.(Eng.) AM Inst. CE and AMI Mech. E. and AMI Struct. E. Examinations in this Subject*, Pitman, 1925.
- [24] R. C. Hibbeler, K.-H. Tan, *Structural analysis*, Pearson Prentice Hall Upper Saddle River, NJ, 2006.
- [25] S. Leyffer, J. Linderoth, J. Luedtke, A. Miller, T. Munson, Applications and algorithms for mixed integer nonlinear programming, in: *Journal of Physics: Conference Series*, Vol. 180, IOP Publishing, 2009, p. 012014.
- [26] N. Beldiceanu, M. Carlsson, S. Demasse, T. Petit, Global constraint catalogue: Past, present and future, *Constraints An Int. J.* 12 (1) (2007) 21–62.
- [27] C. J. Watkins, P. Dayan, Q-learning, *Machine learning* 8 (3) (1992) 279–292.
- [28] B. J. Kröse, Learning from delayed rewards, *Robotics and Autonomous Systems* 15 (4) (1995) 233–235.
- [29] R. S. Sutton, A. G. Barto, et al., *Reinforcement learning: An introduction*, Vol. 1, MIT press Cambridge, 1998.
- [30] Y. Sun, W. Shi, Y. Gao, A particle swarm optimization algorithm based on an improved deb criterion for constrained optimization problems, *PeerJ Computer Science* 8 (2022) e1178.
- [31] D. Liu, K. C. Tan, C. K. Goh, W. K. Ho, On solving multiobjective bin packing problems using particle swarm optimization, in: *2006 IEEE International Conference on Evolutionary Computation*, IEEE, 2006, pp. 2095–2102.
- [32] C. Liang, X. Liu, The research of collision detection algorithm based on separating axis theorem, *Int. J. Sci* 2 (10) (2015) 110–114.
- [33] K. Deb, An efficient constraint handling method for genetic algorithms, *Computer methods in applied mechanics and engineering* 186 (2–4) (2000) 311–338.
- [34] N. Nethercote, P. J. Stuckey, R. Becket, S. Brand, G. J. Duck, G. Tack, Minizinc: Towards a standard cp modelling language, in: *International Conference on Principles and Practice of Constraint Programming*, Springer, 2007, pp. 529–543.
- [35] C. Schulte, M. Lagerkvist, G. Tack, Gecode, Software download and online material at the website: <http://www.gecode.org> (2006) 11–13.
- [36] G. Chu, P. J. Stuckey, A. Schutt, T. Ehlers, G. Gange, K. Francis, Chuffed: A lazy clause generation solver, <https://github.com/chuffed/chuffed>, accessed: 2025-05-27 (2018).
- [37] Google LLC, OR-Tools: Google’s operations research tools, <https://github.com/google/or-tools>, accessed: 2025-05-27 (2024).
- [38] Gurobi Optimization, LLC, Gurobi Optimizer Reference Manual (2025). URL <https://www.gurobi.com>
- [39] D. Pisinger, S. Ropke, Large neighborhood search, in: *Handbook of metaheuristics*, Springer, 2018, pp. 99–127.
- [40] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2623–2631.